Supplementary Material: When Agent Failure Prediction Measures Task Difficulty Rather Than the Run

ANONYMOUS AUTHOR(S)

This supplement collects derivations and full tables referenced in the main paper. Exhaustive per-cell grids, all seeds, and a `make reproduce-main` pipeline are in the replication package (https://github.com/seanonymoussubmission/agent-monitor-estimand); each item below names the script that produces it.

1 DERIVATIONS

1.1 The pooled decomposition (main §2.3)

Pooled AUROC is the concordance probability over positive–negative (failure–success) pairs. Partitioning pairs into same-unit and cross-unit gives the identity

$$A_{\text{pool}} = w\,A_{\text{within}} + (1-w)\,A_{\text{cross}}, \qquad w = \frac{\sum_u P_u N_u}{\left(\sum_u P_u\right)\left(\sum_u N_u\right)},$$

with P_u, N_u the failing/succeeding runs of unit u. It holds for any scoring function; we verify it to 10^{-9} in every reported configuration. For n units of r runs, same-unit pairs number $O(nr^2)$ against $O(n^2r^2)$ cross-unit, so $w = \Theta(1/n)$; measured values are 2.5×10^{-4} (C1), 3.4×10^{-5} (C2), 8.8×10^{-5} (C3), 6.8×10^{-4} (C4-L).

1.2 The difficulty-oracle ceiling (main §2.3)

Score runs by the unit failure rate q_u. A drawn failure comes from unit a with weight $\propto r_a q_a$, a drawn success from b with weight $\propto r_b(1-q_b)$. Writing $q_a(1-q_b) = \frac{1}{2}[(q_a+q_b-2q_aq_b)+(q_a-q_b)]$, the symmetric term integrates to $\mu(1-\mu)/2$ over the ordered half and the antisymmetric term to MD/4 (MD $= \mathbb{E}|q_a - q_b|$, the Gini mean difference), giving

$$A^{\text{oracle}}_{\text{cross}} = \tfrac{1}{2} + \frac{\text{MD}}{4\mu(1-\mu)}, \qquad w = \frac{\overline{q(1-q)}}{n\,\mu(1-\mu)}.$$

Both are computable from the outcome table alone. Predicted vs. measured leave-one-run-out oracle: 0.954/0.945 (C1), 0.987/0.974 (C2), 0.984/0.903 (C3), 0.973/0.949 (C4-L), 0.979/0.959 (C4-Q). Script: `90_predict_inflation.py`.

1.3 Finite-run refinement (main §2.3)

Conditioning on a drawn run's own label leaves $r-1$ i.i.d. Bernoulli(q_u) runs, so both a drawn failure's and a drawn success's leave-one-out score are $\text{Bin}(r-1, q_u)/(r-1)$. The LOO oracle's cross-unit AUROC is the concordance of the failure- vs. success-weighted mixtures of these; same-unit pairs are fully discordant (the anticorrelation artifact of the LOO estimator). This halves the ceiling's error on all five corpora (0.950/0.980/0.940/0.962/0.969 predicted); the residual falls with runs per unit (0.037 at C3's three, 0.005–0.013 at ten or more) and is plug-in dispersion bias. Script: `91_finite_r_theory.py`.

1.4 Analytic permutation-null SD (main §2.5)

Under H_0 the within-unit Mann–Whitney statistic has exact null variance $(P_u+N_u+1)/(12P_uN_u)$; the pair-weighted aggregate has null SD $\sqrt{\sum_u P_uN_u(P_u+N_u+1)/12}\,/\sum_u P_uN_u$. Predicted vs. empirical:

Table 1. C4-L activation probe, within-task AUROC by prefix k and layer (142 mixed units). No cell survives Holm; best k=10, L30 (0.546, $p_{\text{Holm}} = 0.40$). Script: `24_lph_analysis.py`.

	L10	L20	L30	L40
k=1	0.482	0.524	0.507	0.521
k=3	0.514	0.517	0.478	0.505
k=5	0.488	0.527	0.516	0.490
k=10	0.546	0.536	0.546	0.520

0.0058 vs. 0.0057 (C2), 0.0183 vs. 0.0177–0.0190 (C3), 0.018 vs. 0.018 (C4-Q), 0.020 vs. 0.021 (C4-L probe subset). Script: `92_analytic_null.py`.

1.5 Fluid-limit allocation anchor (main §6.2)

With known parameters the expected cost to solve u is $\bar{c}_u/\theta_u$; serving arms greedily by $\theta_u/\bar{c}_u$ maximises solves under budget in the fluid limit—a ratio-index policy (cf. Smith's rule for weighted completion time). The oracle implements it and Thompson sampling approximates it in posterior form. Arm retirement on success is endogenous, outside the standard budgeted- and mortal-bandit analyses, so no regret guarantee is claimed.

1.6 Look-ahead leakage proposition (main §4.5)

If the cut index $k = g(T)$ is an *injective* function of eventual length T, then observing the prefix reveals k and hence T; any measurable function of T is then a realisable score, so attainable within-task AUROC is lower-bounded by $\text{AUROC}(T) = 0.614$ (measured, C1). A fractional cut $k = \lfloor \alpha T \rfloor$ is instead a *coarsening* of T: it need not identify T exactly, so it carries at most this much length information and in general strictly less—the injective case is an upper bound on the leak, not a guarantee it is attained. Empirically on C1 the coarsened leakage already accounts for most of the apparent within-task discrimination (+0.0616 at matched depth). For a fixed index k, prefix statistics are functions of the first k turns only and carry no such dependence on the future.

1.7 Finite-repeat ICC correction (main §5.2)

The naive ICC = $\text{Var}(\bar{y}_u)/\text{Var}(y)$ is upward-biased because $\text{Var}(\bar{y}_u) = \text{Var}(q_u) + \mathbb{E}[q_u(1-q_u)/r_u]$ carries within-unit sampling noise. We correct two ways. (i) A one-way random-effects ICC(1), $(\text{MSB} - \text{MSW})/(\text{MSB} + (n_0 - 1)\text{MSW})$ with $n_0 = (N - \sum_u r_u^2/N)/(k-1)$. (ii) A beta-binomial method of moments using the unbiased per-unit estimator $\widehat{q_u(1-q_u)} = s_u(r_u - s_u)/(r_u(r_u-1))$, giving $\text{Var}(q) = \mu(1-\mu) - \mathbb{E}[q(1-q)]$ and ICC = $\text{Var}(q)/(\mu(1-\mu))$. On C3 (mean $r = 3$) the naive 0.800 falls to 0.699 under both corrections (they agree to 10^{-3}), 95% unit-bootstrap CI $[0.68, 0.72]$: between-unit heterogeneity still dominates. Script: `97_icc_corrected.py`.

2 FULL TABLES

Capability-legibility robustness (main, depth-and-capability analysis). The C3 capability trend ($\rho = -0.662$) survives every check we ran. Permutation: shuffling capability labels across the 17 models gives a null centred at +0.0002 (sd 0.247), exact $p = 0.0046$. Confounds: partialling out outcome-blind behavioural dispersion moves the estimate only $-0.662 \to -0.668$. Dependence: a model-resample bootstrap that accounts for tasks shared across models gives a 95% CI of $[-0.90, -0.23]$, and a precision-weighted slope is -0.674 ± 0.113. Measurement: capability computed

Table 2. C3 factorial: pre-execution predictors, held-out cross-fitted by task (within = 0.5 by construction). Script: `93_factorial_and_taskboot.py`.

Predictor	pooled AUROC
model identity only	0.568
task only (difficulty × domain)	0.826
task × model	0.845
unit oracle (leave-one-run-out)	0.903
per-model task oracle (median; range)	0.985 (0.962–0.998)

Table 3. Operating curve, C4-L k=5 automaton monitor: abort when score $\geq \tau$. Script: `94_operational.py`.

success retention	false-abort	failure recall	compute saved
0.99	0.01	0.014	0.012
0.97	0.03	0.054	0.043
0.95	0.05	0.091	0.073
0.90	0.10	0.139	0.120

Table 4. Label-noise calibration (C4-L): observed within-task AUROC after corrupting a fraction α of success labels, for synthetic signals of three true effect sizes. Script: `81_noise_calibrated.py`.

α	true 0.55	true 0.60	true 0.65
0.00	0.542	0.614	0.664
0.05	0.529	0.577	0.620
0.10	0.521	0.560	0.600
0.20	0.514	0.542	0.565
0.30	0.507	0.528	0.547
0.50	0.508	0.524	0.539

Table 5. Bandit allocation, C4-Q target (prior transferred from C4-L), distinct tasks solved by token budget; 200 trials (100 for the static row). The static ranking walks the prior order in fixed priority (a failed task waits for the next pass); a greedy variant that retries the top-ranked task until it succeeds collapses to 208.0 flat. Script: `54_bandit_allocation.py`.

Policy	2M	5M	10M	40M
Round-robin	53.1	131.3	261.7	360.3
Static ranking, transferred prior	166.6	256.9	295.8	360.1
Bandit, uninformative prior	135.4	212.7	269.9	358.2
Bandit + transferred prior	**162.5**	**246.0**	**307.9**	360.5
Oracle (true θ_u)	172.3	277.2	345.6	375.0

on disjoint tasks and on disjoint domains, matched within-unit pair counts, and leave-one-family-out all preserve the negative sign. Script: `72_behavioral_dispersion.py`, `74_within_family.py`, `85_dependence.py`.

Table 6. Cold-start prior seeded from pre-execution features, novel-task regime (C4-L), tasks solved at scaled budget vs. pseudo-count weight. Script: `68_coldstart_prior.py`.

prior	tasks solved
uninformative	108.99 ± 4.45
pre-execution seed, λ=0.25	108.63
λ=1.0	107.35
λ=4.0	102.36
λ=8.0	96.90
transferred (rollout-derived)	124.74
oracle	139.10

Table 7. Original full-pool C4-L comparison (500 tasks, 150 replay trials), kept for reference: the turn cap and the median-threshold abort demoted from the main table (main, abort-vs-reallocate analysis). Allocate here seeds its prior from the replayed tasks' own outcome counts, which the main table's fair protocol replaces with a cross-model transferred prior on a held-out split. Scripts: `79_abort_vs_allocate.py`, `98_deployment_fair.py`.

Budget	Round-robin	Turn-cap 10	Abort@5 (median)	Allocate	Hybrid
2M	30.5	2.6	34.9	109.5	104.2
5M	75.8	5.8	87.1	194.4	181.0
10M	151.3	9.2	165.8	276.8	248.9
20M	291.2	12.4	236.8	332.8	292.8

Table 8. C1 depth sweep on a fixed cohort of 63,657 traces (885 mixed units, identical at every k), automaton featuriser (main, depth-and-capability analysis). The varying-sample column is what a naive sweep reports; the two agree, so selection is not driving the rise. Script: `66_characterize.py`.

k	varying sample	fixed cohort (95% CI)	
1	0.5072	0.5088	[0.4982, 0.5189]
3	0.5114	0.5105	[0.4984, 0.5211]
5	0.5386	0.5320	[0.5214, 0.5428]
10	0.5858	0.5858	[0.5717, 0.5988]

3 SECOND REPLAY EXPERIMENT: UNIT-LEVEL VS RUN-LEVEL VALUE OF THE SCORE

How the score is used matters more than whether, and the accounting is cost-honest by construction: a k=5 score exists only for runs the policy has drawn and paid for in full—no score is acquired without its run. A second replay experiment (120 trials on the full task pool; own baselines: round-robin 30.9, Allocate 322.3 at 2M/20M—slightly below the supplement's full-pool comparison because this run fixes the prior weight at 0.5) separates the score's unit-level from its run-level value, and corrects a reading we ourselves initially found tempting. Layering each paid pull's score onto cold outcome-Thompson as difficulty pseudo-evidence *hurts* (79.2 vs 87.5 solved at 2M; 116.7 vs 149.9 at 5M), and routing on monitor evidence *alone*—no outcome memory at all—cannot allocate: 28.3/28.4 at 2M/5M, below round-robin and flat in budget, because the policy revisits monitor-favoured unsolvable tasks that only an observed failure would demote. Even at cold start, one real outcome is worth more than the score. The same monotonicity holds with a prior: monitor input layered on the transferred prior never beats this experiment's Allocate (320.2 vs 322.3

Table 9. Within-unit concordance by capability tercile on C3, each behavioural family fitted alone (main, depth-and-capability analysis). Weak models are legible through several channels; for stronger models none clears chance. Script: `73_signal_source_by_capability.py`.

Family	weak	middle	strong
All families	**0.666**	0.521	0.529
Transition surprise	**0.638**	0.508	0.483
Output size	**0.610**	0.504	0.484
Which actions	**0.609**	0.534	0.487
Repetition	0.534	0.474	0.477
Errors seen	0.529	0.470	0.501

Table 10. Abort-and-restart deployment simulation on C1 (main, abort-vs-reallocate analysis): a restart redraws the outcome from the same exchangeable unit; baseline failure rate 0.8334. Δsuccess, disruption/recovery ratio d/r, and break-even range span all operating points. The predictor selected by A_{pool} is negative at all seven points swept; the one selected by A_{within} is non-negative at all twelve. Script: `79_abort_vs_allocate.py`.

Predictor (selection metric)	Δsuccess	d/r	break-even
Oracle (A_{pool})	−0.0012 to −0.0028	14.7–348	0.936–1.000
Trajectory (A_{within})	+0.0000 to +0.0029	5.1–21.8	0.835–0.956

Table 11. Within-task discrimination at matched prefix depths (main §5.5). First three rows use the automaton behavioural featuriser; the last is the best of four hidden-state probe layers per depth (generous to the probe).

	k=1	k=5	k=10	full
C1 (Llama 8B–405B)	0.509	0.532	0.586	**0.688**
C2 (single stronger model)	0.504	0.501	0.503	—
C4-L, behaviour	0.501	0.496	0.514	**0.656**
C4-L, activations	0.524	0.527	0.546	n/a

at 20M at the lightest weight) and hurts as trusted more (to 142 at weight 1), while a *fair* abort (break-even of §**??**, calibration favouring it) recovers nearly all the naive loss yet does not beat allocation (317.0 vs 322.3)—"aborting adds nothing" is conservative. In short: do not route on it; calibrate any abort to break-even; never let it override observed outcomes.

4 AUTOMATA REIMPLEMENTATION DETAILS

Activity abstraction over commands; Last-Activity-Merge FSM with rare-transition filtering; gradient boosting over per-state behavioural plus cross-entropy anomaly features; transition model fit on training folds only. Our FSMs have 35–38 states, inside the paper's reported 7–43. At the 25% checkpoint the original reports rank-AUROC 0.66 while we obtain pooled 0.728; these are different statistics, so no reproduction is claimed at that point. Script: `63_automata_reeval.py`.

5 VERIFICATION PROTOCOL

Every probe result passes four gates (main §5.3): (1) weight integrity—zero missing keys and a teacher-forced accuracy floor of 0.55 (one code path silently randomised all 39 MoE layers; Apple-MPS loaded correctly yet computed wrong values); (2) replay from stored token ids, never a

Table 12. Property map of the surveyed trajectory-level failure-prediction literature (descriptive, not a uniqueness claim; full citations in the main paper's reference list). *Pfx*: operates on a partial trajectory. *Rep*: uses repeated runs of the same model on the same task. *Nat*: failures occur naturally rather than being injected. *≥5 env.*: the paper's main results span five or more distinct task suites, counting a benchmark or a separately-harnessed domain as one environment. *Dep*: measures the effect of acting on the prediction (∼ for EarlyEval and TRACER, whose halting/selection targets benchmark efficiency or offline selective execution rather than attempt recovery). ∼ denotes partial satisfaction; n.r. not reported.

Paper	Pfx	Rep	Nat	≥5 env.	Dep
Who&When	×	×	✓	×	×
MAST	×	×	✓	✓	×
TRAIL	×	×	✓	×	×
AgentRewardBench	×	×	✓	✓	×
AgenTracer	×	×	×	✓	∼
DRIFT	×	×	✓	×	×
Argus	×	×	✓	✓	×
Psychometrics	×	∼	✓	✓	×
Failure as Process	∼	×	✓	×	×
Automata	✓	×	✓	✓	×
Doomed	✓	✓	✓	×	∼
Last Step	✓	∼	✓	×	∼
Capable	✓	✓	✓	×	×
Paradox	✓	n.r.	✓	×	✓
TRACER	n.r.	×	✓	×	∼
FailFast-RestartSmart	✓	n.r.	✓	×	✓
Real-Time D&R	✓	n.r.	✓	×	✓
EarlyEval	✓	×	✓	×	∼
This paper	✓	✓	✓	✓	✓

Table 13. Difficulty-only ceilings computed from public leaderboard aggregates alone (main §5.4). τ^2-bench submissions publish pass^1..pass^4 per domain ($\geq$ 4 trials); these are the first four moments of the per-task success probability, so a beta method-of-moments fit yields the ceiling of main §2.3 with no model, scores, or trajectories. Fit quality: fitted third/fourth moments match the published ones to 0.003–0.024. "LOO@4" is the Monte-Carlo leave-one-out difficulty oracle at the benchmark's own four trials. Script: `101_leaderboard_ceiling.py`.

Domain	Model	μ_{fail}	mixed@4	MD	ceiling	LOO@4
telecom	gpt-5-2	0.103	0.271	0.135	0.865	0.708
telecom	gemini-3-flash	0.088	0.298	0.055	0.671	0.533
telecom	glm-5-think	0.132	0.377	0.103	0.726	0.589
retail	gpt-5-2	0.184	0.448	0.186	0.810	0.691
retail	gemini-3-flash	0.233	0.394	0.279	0.891	0.798
retail	glm-5-think	0.263	0.491	0.270	0.848	0.750

reconstructed template; (3) an independent estimator implementation agreeing to 10^{-9}; (4) positive controls—an outcome-carrying feature recovers within-task 1.000, Gaussian noise 0.503, and real activations with shuffled labels 0.488. Scripts: `27_load_integrity.py`, `61_validate.py`.

Table 14. Pre-registered experiment E1: within-unit training objective on C1 (926 mixed units; same features and GroupKFold-by-task folds for every objective). "Ranker" is a same-unit pairwise Bradley–Terry (the conditional-logit estimator for pairs); "demeaned" fits pooled logistic on within-unit demeaned features (test-time demeaning uses the unit's other runs' features, no labels—a diagnostic, not deployable). Cohort: all parseable C1 traces, behavioural+anomaly features—a different instrument from the main text's fixed ≥10-turn cohort, so the two pooled-trained baselines differ slightly. All ranker/demeaned within values reject H_0=0.5 (permutation $p < 0.002$, the 600-permutation floor; null sd 0.004); controls: injected outcome feature → within 1.000, shuffled labels → 0.491. Decision rule pre-registered in `PREREGISTRATION.md`. Script: `99_within_objective.py`.

	pooled objective		ranker		demeaned	
k	pooled	within	pooled	within	pooled	within
1	0.529	0.504	0.517	0.526	0.501	0.526
3	0.613	0.509	0.533	0.535	0.506	0.533
5	0.649	0.537	0.558	0.547	0.517	0.546
10	0.700	0.583	0.641	0.596	0.542	0.595

Table 15. E1-C2: the same three objectives on C2 (SWE-rebench; 1,741 mixed units, behavioural features, permutation null sd ≈ 0.006). Every cell is a null (all $p > 0.057$): the flat C2 result is fully objective-independent. Script: `103_within_objective_c2.py`.

	pooled objective		ranker		demeaned	
k	pooled	within	pooled	within	pooled	within
2	0.534	0.504	0.528	0.503	0.490	0.496
5	0.530	0.502	0.492	0.501	0.490	0.504
10	0.538	0.511	0.515	0.503	0.491	0.505
20	0.568	0.503	0.507	0.502	0.498	0.504

Table 16. E1-P: within-objective *probes* on the C4-L archive (96 mixed units; same folds and standardization for both objectives). Cell = within-task AUROC of the same-unit ranker (pooled-objective value in parentheses). No cell approaches the pre-registered 0.58 bar; shuffled-label control 0.503. Script: `102_within_objective_probe.py`.

	L10	L20	L30	L40
k=1	0.470 (0.504)	0.473 (0.550)	0.479 (0.502)	0.533 (0.536)
k=3	0.569 (0.550)	0.553 (0.516)	0.546 (0.524)	0.502 (0.482)
k=5	0.455 (0.508)	0.526 (0.529)	0.460 (0.534)	0.460 (0.491)
k=10	0.531 (0.510)	0.519 (0.530)	0.494 (0.488)	0.545 (0.541)

6 LITERATURE-SEARCH PROTOCOL

Four Google Scholar query families (failure attribution; LLM-agent failure prediction; agent-trajectory failure; coding-agent failure), 2023 onward; every returned trajectory-level failure-prediction result was read and snowballed, and each load-bearing claim verified against the cited paper's own text. The citation-knee stopping rule is inapplicable to a literature whose 2026 works have zero citations, so we substituted an exhaustive read of the returned set.

Table 17. M-r: standardized mixed-outcome rates at a common repeat count, exact hypergeometric over units with $n \geq r$. "Raw" is the share of all units. Script: `104_standardized_mixedness.py`.

Corpus	units	raw	std@r=3	std@r=5
C1	4,219	0.220	0.110	0.155
C2	6,306	0.276	0.152	0.208
C3	2,278	0.219	0.219	—
C4-L	500	0.406	0.214	0.298
C4-Q	500	0.348	0.188	0.259

Table 18. Pre-registered experiment E2: required within-task AUROC. The fair replay harness of the main deployment table, with the monitor replaced by synthetic scores of controlled within-unit discrimination and zero between-unit component (achieved within verified per table; thresholds tuned per budget on the tuning split). Entries: paired tasks-solved difference, tuned hybrid minus allocation, mean [95% CI]; bold = CI excludes zero. Even a perfect monitor adds 2.6–7.8 tasks. Two generosities make the crossing a *lower* bound on the true requirement: the synthetic score is unit-demeaned (zero between-unit component), and a global threshold on a demeaned score acts as a within-unit-relative abort that a single-run online monitor cannot compute. Script: `100_required_within.py`. Family B (pre-registered amendment) repeats the sweep with a task-level component added to the score (transferred-prior failure rate, weight 3; achieved pooled 0.87–1.00, no demeaning): the hybrid–allocation crossings do not move materially (CIs span zero throughout within ≤ 0.70 despite pooled ≈ 0.9; first clear gain at within 0.926/5M, +14.5 [+6.7, +24.3]), confirming the pre-registered prediction—while abort-only over round-robin turns uniformly positive (+11 to +75 at every level, even within 0.549): a task-aware abort performs allocation-by-truncation, roughly matching plain allocation at 5M. The task signal carries the value whichever channel exploits it.

achieved A_{within}	1M	2.5M	5M	10M
0.549	−0.8 [−7.3, +5.0]	−2.8 [−12.3, +7.0]	−1.2 [−9.0, +8.0]	−2.4 [−11.3, +6.0]
0.634	−1.1 [−7.0, +5.0]	−1.6 [−10.0, +6.3]	−0.3 [−10.3, +9.0]	+2.4 [−6.5, +10.0]
0.651	−0.0 [−6.0, +6.0]	+1.5 [−7.3, +12.0]	+4.6 [−4.0, +14.0]	+3.7 [−5.3, +11.3]
0.694	−0.2 [−6.0, +6.0]	−0.3 [−9.3, +7.0]	+2.8 [−6.0, +11.3]	+3.5 [−5.0, +12.0]
0.838	−1.3 [−7.3, +4.3]	+0.5 [−7.3, +9.0]	+6.1 [−3.0, +14.3]	**+8.0** [+0.7, +15.3]
0.926	+0.0 [−5.3, +5.3]	+2.8 [−5.3, +11.0]	**+8.7** [+0.7, +16.3]	**+9.4** [+3.0, +16.3]
1.000	+2.6 [+0.0, +6.3]	**+5.8** [+1.0, +11.0]	**+7.8** [+3.0, +13.0]	**+5.8** [+1.0, +11.0]